

Exam III, Theory of Computation 2018-2019

- Books, notes, communication, calculators, cell phones, computers, etc... are not allowed.
- Your explanations and proofs should be clear enough and in sufficient detail so that they are easy to understand and have no ambiguities.
- You are allowed to refer to material covered in the class including theorems without reproving them.
- **Do not touch until the start of the exam.**

Good luck!

Name: _____ N° Sciper: _____

Problem 1	Problem 2	Problem 3	Problem 4	Problem 5	Problem 6
/ 8 points	/ 6 points	/ 8 points	/ 6 points	/ 6 points	/ 6 points

Total / 40

1 **Basic questions.** (8 pts) Let $A \subseteq \{0,1\}^*$ and $B \subseteq \{0,1\}^*$ be two languages. Which of the following statements are true?

1. $A \leq_m B \implies A \leq_p B$.
2. If $\mathbf{P} = \mathbf{NP}$, then $\{\langle \phi \rangle \mid \phi \text{ is a satisfiable CNF formula}\}$ is in $\mathbf{P}$.
3. $\{\langle M, w \rangle \mid M \text{ is a TM and } M \text{ doesn't halt on input } w\}$ is in $\mathbf{NP}$.
4. A is recognized by a DFA $\iff A$ is recognized by an NFA.
5. A is regular and $A \leq_m B \implies B$ is regular.
6. A is regular $\implies A$ is decidable $\implies A$ is in $\mathbf{P} \implies A$ is in $\mathbf{NP}$.
7. A is in $\mathbf{P}$ and $B \leq_p A \implies B$ is in $\mathbf{P}$.
8. $A \in \text{TIME}(n^{100}) \implies A \in \mathbf{NP}$.
9. Suppose A is $\mathbf{NP}$ -hard and B is $\mathbf{NP}$ -complete. If B is in $\mathbf{P}$ then A is in $\mathbf{P}$.
10. A is in $\mathbf{P}$ and B is in $\mathbf{P} \implies A \circ B$ is in $\mathbf{P}$.
11. A is recognizable by a TM $\implies$ its complement $\bar{A}$ is recognizable by a TM.
12. $\{w \in \{0,1\}^* \mid \text{number of 1's in } w \text{ is divisible by } 2019\}$ is regular.

(A complete solution identifies **all** true statements. A fully correct solution is worth 8 points. A solution with one mistake is worth 7 points. A solution with two mistakes is worth 5 point. A solution with three mistakes is worth 3 points. A solution with four mistakes is worth 1 point. Solutions with more mistakes are worth 0 points. A mistake is to either indicate falsely that a false statement is true or to not indicate that a true statement is true.)

Solution:

2, 4, 7,8, 10, 12

2 Regular languages. (6 pts) Consider the following language on alphabet $\Sigma = \{0, 1\}$:

$$L_2 = \{w \in \{0, 1\}^* \mid w \text{ has more 0's than 1's}\}.$$

Is L_2 regular? Provide a proof of your claim.

*(In this problem you should decide whether L_2 is regular or not, and to give a **proof** of your claim. Recall that you are allowed to refer to material covered in the class including theorems without reproving them.)*

Solution: L_2 is a non-regular language. We will prove our claim by contradiction.

We use the Pumping Lemma. Suppose that L_2 is a regular language and that its pumping length is p . Consider the string $s := 1^p 0^{p+1} \in L_2$. By the Pumping Lemma, s can be decomposed as $s = xyz$ where $|xy| \leq p$, $|y| \geq 1$, and $xy^i z \in L_2$ for all $i \in \mathbb{N}$. Since s starts with p ones, $y = 1^k$ for some integer k , $1 \leq k \leq p$. Then $xy^2 z = 1^{p+k} 0^{p+1} \in L_2$ by the Pumping Lemma. However, $p+k \geq (p+1)$ for all $1 \leq k \leq p$, which contradicts that $xy^2 z \in L_2$. Hence, L_2 is not regular.

- 3 Graph coloring.** (8 pts, consisting of subproblems (a) and (b)) A coloring of an undirected graph is an assignment of colors to its vertices so that no two adjacent vertices are assigned the same color. For each positive integer k , let

$$k\text{-COL} = \{\langle G \rangle \mid G \text{ is colorable with } k \text{ colors}\}.$$

In Homework 3 you proved that 3-COL is **NP**-complete. In this problem we will consider the related languages 2-COL and 4-COL. Specifically, we will show that 2-COL is in **P** whereas 4-COL is **NP**-hard.

- 3a** (3 pts) Show that 2-COL is in **P**.

(In this problem you should show that 2-COL is in **P**. Recall that you are allowed to refer to material covered in the class including theorems without reproving them.)

Solution: The following algorithm decides 2-COL:

Algorithm 1 Is2COL($G = (V, E)$)

```

procedure DFS( $G, w, col$ )
    color( $w$ ) = col
    for  $u \in N(w)$  do
        if color( $u$ ) == color( $w$ ) then
            return FALSE
        if color( $u$ ) ==  $\perp$  then
            DFS( $G, u, col \oplus 1$ )
    return TRUE
procedure Is2COL( $G = (V, E)$ )
    for  $v \in V$  do
        color( $v$ ) =  $\perp$ 
    for  $v \in V$  do
        if color( $v$ ) ==  $\perp$  and DFS( $G, v, 0$ ) == FALSE then
            return FALSE
    return TRUE

```

First notice that the following algorithm runs in time $O(|V| + |E|)$ as the algorithm is basically running a DFS. The algorithm visits each vertex and each edge exactly twice.

Moreover notice that in any correct 2-coloring for each edge its endpoints have to be colored with opposite colors. So for each connected component if we color one vertex, let's call it v , then if correct coloring of this component exists it is uniquely defined by color assigned to v (neighbours of v are colored with opposite color to v , neighbours of neighbours are colored with the same colors as v , ...). Algorithm 1 uses that fact, for each connected component it picks a starting vertex v , colors it with color 0 and then explores this connected component using a DFS algorithm.

At each point of the algorithm let $S \subset V$ be the set of vertices with assigned colors (which is exactly the set of all explored vertices). At each step the algorithm looks at an edge (w, u) , $w \in S$. If $u \notin S$ then set S grows by u and u is assigned the only color consistent with the edge (u, w) and the previous coloring. If $u \in S$ then the Algorithm checks if the so far built coloring is consistent with edge (u, w) . So at each point of the algorithm the following invariant is satisfied: color(S) is a valid coloring for graph $G' = (S, E')$, where $E' \subset E$ is a set of visited edges or G is not 2-colorable and Algorithm has returned FALSE. Properties of DFS guarantee that it will visit

all the edges and that set S is always connected within one component. These facts guarantee correctness of the Algorithm as if G is 2-colorable the Algorithm will find the right coloring and if it's not it will detect that fact.

3b (5 pts) Show that $3\text{-COL} \leq_p 4\text{-COL}$.

(In this problem you should describe a polynomial time reduction from 3-COL to 4-COL and prove its correctness. Recall that you are allowed to refer to material covered in the class including theorems without reproving them.)

Solution: Given an instance $G = (V, E)$ of 3-COL, create the following instance $G' = (V', E')$ of 4-COL where

$$V' = V \cup \{u\} \quad (\text{add a new vertex } u)$$

$$E' = E \cup \{uv \mid v \in V\} \quad (\text{add an edge from } u \text{ to every vertex in } V).$$

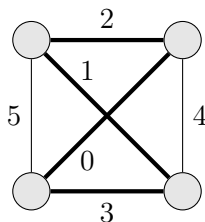
This reduction is polynomial since we added 1 vertex and n edges, where $n = |V|$.

To prove correctness of the reduction:

Suppose $G \in 3\text{-COL}$. Then a valid 4-colouring of G' is to assign to $v \in V'$ the same colour as v in the 3-colouring of G , if $v \in V$. Then we can assign the fourth colour to u and so $G' \in 4\text{-COL}$.

Now suppose $G' \in 4\text{-COL}$. Since there is an edge from u to every other vertex in G' , these vertices must be a different colour from u in a valid 4-colouring. So G can be coloured with 3 colours and hence $G \in 3\text{-COL}$.

- 4 **The traveling salesman problem.** (6 pts) An instance of the classic traveling salesman problem is defined by a tuple $H = (V, d)$ where V is a set of cities and $d : V \times V \rightarrow \mathbb{Z}$ defines their pairwise distances. That is for two cities $i \in V$ and $j \in V$, $d(i, j) = d(j, i)$ equals the distance required to travel between cities i and j . Notice that H can be thought of as a complete graph (every possible edge is present) with weights on the edges corresponding to the distances. A tour is a cycle that visits every city *exactly* once, i.e., it is a cycle of length $|V|$. The length of a tour is the total distance travelled which equals the sum of the distances of the edges in the tour. An example is depicted below. The instance has 4 cities and the thick edges depict a tour of length $0 + 2 + 1 + 3 = 6$.



Show that the following language is **NP**-complete:

$$\text{TSP} = \{ \langle H = (V, d), k \rangle \mid H \text{ has a tour of length at most } k \}.$$

In your proof you may use that the following language is **NP**-complete:

$$\text{HAM} = \{ \langle G \rangle \mid G \text{ is an undirected graph that is Hamiltonian} \}.$$

An undirected graph $G = (V, E)$ is said to be Hamiltonian if it contains a cycle of length $|V|$, i.e., a cycle that visits every vertex. Two examples of Hamiltonian graphs are the complete graph and the graph consisting of a single cycle of length $|V|$.

*(In this problem you are asked to prove that the language TSP is **NP**-complete. Recall that you are allowed to use that HAM is **NP**-complete and to refer to material covered in the class including theorems without reproving them.)*

Solution:

For proving the NP completeness of the TSP problem we have to prove that: (1) the problem is in NP (2) the problem is NP-Hard.

We first prove that the problem belongs in NP. Given a certificate C , which we assume that is an ordered set of edges (the assumption is not restrictive, that means that it can either be a list of edges or a list of vertices) we have to check the following:

- The set of edges form a cycle
- Every vertex is visited
- Every vertex is visited at most once
- the sum of the weights of the edges is at most k

These four properties can be checked in parallel. Indeed while traversing the presumed cycle we sum the weight of every new encountered edge, at the same time we keep a counter for every vertex in the graph (all counters were initialized to zero), in this manner a counter value of 0,1,2

means that the vertex was not visited, visited once, visited twice respectively. The traversing of the set of edges has to end by encountering a vertex with counter equal to 1. At any point in time, if one of the four properties stop to hold we stop and output False, otherwise True. Note that, since one of the four properties will break if the set of edges has a cardinality bigger than $|V|$ that means that the running time of the verifier is $O(|V|)$.

We continue by proving the NP-hardness of TSP, by reducing the Hamiltonian graph problem to TSP. We take as input a graph G and we construct a complete graph G' with the same set of vertices and the following weights on the edges:

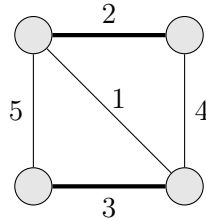
$$\begin{aligned} d(i, j) &= 0 & \forall (i, j) \in G(E) \\ d(i, j) &= \infty & \forall (i, j) \notin G(E) \end{aligned}$$

We set $k = 0$.

Now we will prove that G is Hamiltonian iff G' has a tour of weight at most k

- $\Rightarrow$ If graph G is Hamiltonian then the same tour in G' will be formed from edges of weight 0. Thus the tour will weight 0.
- $\Leftarrow$ If G' has a tour of at most length 0, then this tour can only contain edges of weight 0 (if it contains an edge with weight strictly greater than zero, then the total weight will be more than 0). All these edges from the definition of the weight function existed in G , therefore the same tour was present in G , which makes G a Hamiltonian graph.

- 5 **Perfect matchings of a given weight.** (6 pts) Let $G = (V, E, w)$ be an undirected graph with edge-weights $w : E \rightarrow \mathbb{Z}$. A subset $M \subseteq E$ of the edges is a *perfect matching* if every vertex is incident to exactly one edge in M . In other words, the edges in M pair up all the vertices. The weight of a matching M is the total weight of its edges, i.e., $\sum_{e \in M} w(e)$. An example is depicted below. The graph consists of 4 vertices, 5 edges with integer weights, and the thick edges indicate a perfect matching of weight $2 + 3 = 5$.



Show that the following language is **NP**-complete:

$$L_5 = \{ \langle G = (V, E, w), k \rangle \mid G \text{ has a perfect matching of weight } k \}.$$

(In this problem you are asked to show that the language L_5 is **NP**-complete. Recall that you are allowed to refer to material covered in the class including theorems without reproving them. In particular, you are allowed to use any of the **NP**-complete languages that we saw in class.)

Solution: We first need to show that $L_5 \in \text{NP}$.

Given an instance $\langle G = (V, E, w), k \rangle$ and a certificate consisting of edges $M \subseteq E$, the verifier first checks if the sum of weights of edges in M is k and then checks that every vertex v occurs in exactly one edge in M . The first operation takes $|E|$ time and the second operation takes $|V| \cdot |E|$ time. Hence the verifier takes time polynomial in the size of the input and so $L_5 \in \text{NP}$.

To show that L_5 is NP-hard, we perform the following reduction: $\text{SUBSET-SUM} \leq_p L_5$. Given an instance of SUBSET-SUM $\langle S = \{s_1, \dots, s_n\}, t \rangle$, we construct an instance of L_5 as follows: For each number s_i , create 4 vertices $v_{i,1}, v_{i,2}, v_{i,3}, v_{i,4}$. Connect $(v_{i,1}, v_{i,2})$ with an edge of weight s_i ; connect $(v_{i,2}, v_{i,3})$ with an edge of weight 0; connect $(v_{i,3}, v_{i,4})$ with an edge of weight 0; connect $(v_{i,4}, v_{i,1})$ with an edge of weight 0. Let G denote the new graph. Then $\langle G, t \rangle$ is the constructed instance of L_5 . For each s_i , we add 4 vertices and 4 edges to the graph; hence the construction is polynomial time in the size of the input.

Claim 1 If there is a subset S such that $\sum_{j \in S} s_j = t$, then G has a perfect matching of size t .

Proof. For every $i \in S$, choose the following edges as part of the matching: $(v_{i,1}, v_{i,2})$ and $(v_{i,3}, v_{i,4})$. If $i \notin S$, then choose $(v_{i,2}, v_{i,3})$ and $(v_{i,4}, v_{i,1})$ as part of the matching. It can be seen that this subset of edges is a perfect matching as it covers all the vertices. Furthermore, the only edges contributing to the weight of the matching are $(v_{j,1}, v_{j,2})$, for every $j \in S$. Therefore, the weight of the matching is t . $\square$

Claim 2 If G has a perfect matching of size t , then there exist a subset S such that $\sum_{j \in S} s_j = t$, then

Proof. Consider any connected square $\{v_{i,1}, v_{i,2}, v_{i,3}, v_{i,4}\}$. Since there are 4 edges in this square, two of these edges will be included in the perfect matching. It will either be $(v_{i,1}, v_{i,2})$ and $(v_{i,3}, v_{i,4})$, or $(v_{i,2}, v_{i,3})$ and $(v_{i,4}, v_{i,1})$. If $(v_{i,1}, v_{i,2})$ and $(v_{i,3}, v_{i,4})$ is part of the matching add i to S , otherwise don't add it. Since edge $(v_{i,1}, v_{i,2})$ has weight s_i , that means that adding i to S results in adding s_i weight to the subset S . Therefore, the matching of size k implies that the sum of weights of elements in S is also t . $\square$

Since L_5 is in NP and is NP-hard, it is also NP-complete.

6 Decidability. (6 pts) Consider the following language on alphabet $\Sigma = \{0, 1\}$:

$$L_6 = \{\langle M \rangle \mid M \text{ is a TM and the language recognized by } M \text{ is } \mathbf{NP}\text{-complete}\}.$$

Is L_6 decidable? Provide a proof of your claim.

*(In this problem you should decide whether L_6 is decidable or not, and give a **proof** of your claim. Recall that you are allowed to refer to material covered in the class including theorems without reproving them.)*

Solution:

L_6 is undecidable. We will prove this by reduction from A_{TM} .

Reduction: Let us define a computable function f from the inputs of A_{TM} to the inputs of L_6 .

$$f : \langle M, w \rangle \mapsto \langle T_{M,w} \rangle$$

Let us define the Turing machine $T_{M,w}$:

On input x

1. Run M on w .
2. If M accepts w test if $x \in SAT$. If it is, accept.
3. Else reject.

Note that testing if a string is in SAT can be easily done in exponential time.

Analysis:

- If M accepts w , $L(T_{M,w}) = SAT$ which is NP -complete, as seen in class. Therefore $T_{M,w} \in L_6$.
- If M does not accept w , (both if it rejects and if it never halts,) $L(T_{M,w}) = \emptyset$ which is not NP -complete and therefore $T_{M,w} \notin L_6$. Indeed, if $\emptyset$ was NP -complete there would have to be a reduction from SAT to $\emptyset$. This reduction would have to map elements of SAT to elements of $\emptyset$, which is impossible by definition.

(This page is intentionally left blank.)